

Project 2 - Phase 2

SWE and SWE-c multi-agent architecture

Gursparsh Singh Sodhi, Hithaishi Surendra, Jahnvi Seth, Samhitha Harish,
Samudyata Sudarshan Jagirdar

1 Introduction

In Project 2, Phase 2, our objective was to build a multi-agent architecture for software engineering issue resolution and evaluate it on SWE-bench and SWE-bench-C. In Phase 1, we reproduced baseline results using single-shot, no-repository-access prompting and found that the main bottleneck was localization: models could often generate syntactically valid-looking patches, but the patches failed to apply because they were not grounded in the actual repository state. To address this limitation, Phase 2 focuses on a repository-aware, graph-augmented multi-agent pipeline consisting of four specialized agents: a Planner Agent, Localizer Agent, Diagnostician Agent, and Patcher Agent. The Planner interprets the issue and creates a search strategy, the Localizer uses repository tools and a lightweight code dependency graph to identify relevant files and functions, the Diagnostician analyzes the localized code and prepares a structured fix plan, and the Patcher generates a unified diff using real source-code context. Through this phase, we aim to evaluate whether a structured multi-agent workflow with graph-based repository context can produce more reliable patches than direct single-shot inference.

2 Baseline Evaluation on SWE-bench

2.1 Benchmark Overview

SWE-bench [1] is a benchmark designed to evaluate large language models on realistic software engineering tasks in Python repositories. Each instance is derived from a real GitHub issue and its associated pull request, pairing a natural-language problem description with the repository state at a specific `base_commit` and a human-written gold patch. The task is to generate a patch that resolves the issue in the original repository context.

Unlike standard function-level code generation benchmarks, SWE-bench evaluates **repository-level reasoning**. A successful model must identify the correct file(s), understand the surrounding implementation logic, and produce changes that satisfy the corresponding regression tests. Evaluation is performed with an execution harness that checks out the repository at the exact `base_commit`, applies the candidate patch, installs dependencies, and runs the relevant `FAIL_TO_PASS` tests.

For our baseline setting, we focus on the same **single-shot, no-repository-access** setup used in our SWE-bench-C experiments. This allows us to compare performance across Python and C under the same constraint: the model receives only the issue description and must generate a unified diff patch without interactively reading the codebase.

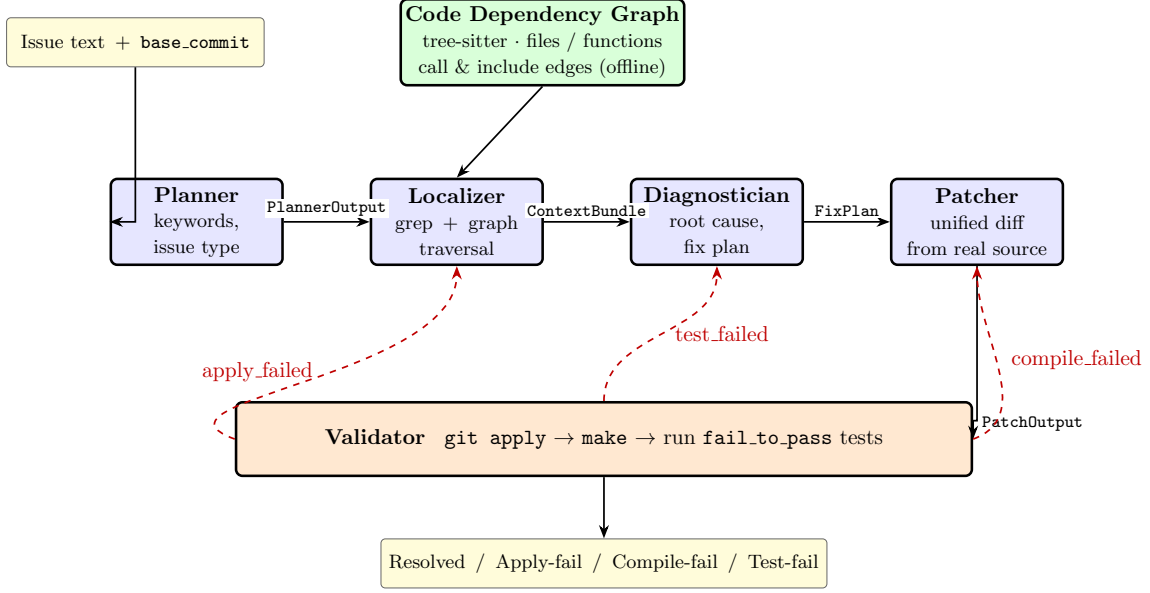


Figure 1: Phase 2 multi-agent pipeline. Four LLM-backed agents (Planner, Localizer, Diagnostician, Patcher) communicate through a typed inter-agent schema, supported by an offline tree-sitter code dependency graph. The Validator runs the SWE-bench-C harness (`git apply` → `make` → `fail_to_pass`) and routes typed failures (red, dashed) back to the agent best able to address each one; every retry slot is capped at two attempts.

3 Experimental Setup

3.1 Benchmark Selection

For Phase 2, we evaluated our proposed multi-agent architecture on two software engineering benchmarks: SWE-bench and SWE-bench-C. SWE-bench evaluates repository-level Python issue resolution using real GitHub issues and pull requests, while SWE-bench-C extends this setting to C repositories. These benchmarks were selected because they require models to perform realistic software engineering tasks, including understanding issue descriptions, identifying relevant source files, modifying code, and producing patches that pass repository-specific tests.

Our Phase 1 results showed that single-shot prompting without repository access was insufficient for both benchmarks. In particular, SWE-bench-C achieved 0% resolution across all runs, with localization failure being the dominant bottleneck. Therefore, Phase 2 was designed to test whether a repository-aware, multi-agent workflow could improve over the Phase 1 baselines by giving agents access to actual source-code context and structured repository information.

3.2 Repository and Dataset Setup

For each benchmark instance, the repository was checked out at the corresponding `base_commit`. This ensured that every generated patch was evaluated against the same repository state used by the benchmark. Each instance consisted of an issue title, issue description, repository metadata, and the benchmark-provided evaluation tests.

For SWE-bench-C, we focused on the three supported C repositories: `facebook/zstd`, `jqlang/jq`, and `redis/redis`. These repositories contain real-world C issues and require the model to produce patches that apply cleanly, compile successfully, and pass the relevant tests.

For SWE-bench, the evaluation followed the Python-based benchmark harness, where generated patches are applied to the repository and tested against the corresponding FAIL_TO_PASS test cases.

3.3 Graph Construction

Before running the agents, we constructed a lightweight code dependency graph for each repository at its `base_commit`. The graph was created using `tree-sitter`, which parses source files without requiring the full project to compile. This was especially useful for SWE-bench-C because C repositories often have complex build systems and cross-file dependencies.

The graph contained both file-level and function-level information. File nodes represented source and header files, while function nodes represented function definitions along with their line ranges. Edges captured structural relationships such as `#include` dependencies, function calls, type or struct usage, and test-to-source relationships. This graph was stored in memory using Python dictionaries and was used by the Localizer Agent to retrieve files that were not only textually relevant to the issue, but also structurally connected through calls, headers, or shared types.

This setup directly addressed one of the main findings from Phase 1: file paths alone were not enough to improve localization. The model needed actual source content and structurally relevant context in order to generate applicable patches.

3.4 Multi-Agent Pipeline

Our Phase 2 system used a four-agent architecture consisting of a Planner Agent, Localizer Agent, Diagnostician Agent, and Patcher Agent.

The **Planner Agent** received only the issue title and description. Its role was to classify the issue as a bug fix, feature request, or refactor, extract important keywords, and generate an initial search strategy for the repository.

The **Localizer Agent** used the Planner’s search strategy along with repository tools such as `grep`, `find`, file reading, and graph-neighbor queries. It searched for relevant functions and files, expanded the search through graph relationships such as callers, callees, headers, and type dependencies, and produced a ranked context bundle containing the most relevant source files.

The **Diagnostician Agent** received the issue description, localized source files, graph context, and relevant test information. Its role was to identify the likely root cause of the issue and produce a structured fix plan. This included the suspected buggy lines, expected behavior, constraints from the tests, and files that required modification.

The **Patcher Agent** received the fix plan and the exact source files to be edited. It generated a unified diff patch using real repository context rather than hallucinated file contents. Since the Patcher had access to the actual source code, it could copy exact context lines and produce patches more likely to pass `git apply`.

3.5 Feedback and Validation

After patch generation, each candidate patch was passed to the benchmark validation pipeline. The validator first checked whether the patch applied cleanly using `git apply`. If patch appli-

cation failed, the error was routed back to the Localizer Agent, since this usually indicated an incorrect file, wrong context lines, or insufficient localization.

If the patch applied successfully, the system attempted compilation or test execution depending on the benchmark. For SWE-bench-C, the patch was compiled using the repository’s build system. Compilation failures were routed back to the Patcher Agent with the compiler error message, allowing the patch to be revised using more precise type, header, or syntax information.

If compilation succeeded but tests failed, the failure output was routed back to the Diagnostician Agent. This allowed the system to revise its understanding of the bug and generate an updated fix plan. Each feedback loop was capped at a maximum of two retries to control cost and runtime.

3.6 Evaluation Metrics

We evaluated the system using the same progressive criteria as Phase 1. For SWE-bench-C, the three main metrics were:

- **Localization:** whether the generated patch applied cleanly using `git apply`.
- **Compilation:** whether the patched repository compiled successfully.
- **Resolution:** whether the patched repository passed the relevant `FAIL_TO_PASS` tests.

For SWE-bench, the main metrics were patch application and final issue resolution through the official evaluation harness. In addition to these benchmark-level metrics, we also recorded intermediate agent-level information such as localization confidence, selected files, feedback-loop triggers, retry counts, and failure types. These logs helped us analyze which stage of the multi-agent pipeline contributed most to improvements or remaining failures.

4 Results

4.1 Results of baseline

This section presents the evaluation results for our Phase 2 experiments on SWE-bench and SWE-bench-C. Since the goal of Phase 2 is to evaluate whether repository-aware multi-agent workflows improve over the Phase 1 single-shot baselines, we report results across progressive stages such as localization, compilation, and final issue resolution.

4.2 Results of SWE-BENCH-AGENT

Table 2 reports the three-stage evaluation of SWE-bench-AGENT against the Phase 1 single-shot baselines on the SWE-bench-C jq subset. SWE-bench-AGENT localised the affected files in 15 instances and produced structurally valid diffs in 9, showing that the evidence-routed pipeline reliably reaches patch generation. The prompt-engineered Qwen3 Enhanced and GPT-4.1-mini baselines achieve higher localisation by biasing the model toward issue-referenced filenames, but their compilation rates collapse to 4 and 0 respectively. No system resolved any instance, consistent with the published 0% Phase 1 baseline on SWE-bench-C.

Table 1: Per-instance Localization / Compilation / Testing results. **S** = Success, **F** = Failed, **N/A** = stage not reached.

Instance ID	Paper (MSWE-agent) [2]			V1: Qwen3 Blind			V2: Qwen3 Enhanced			V3: GPT-4.1-mini		
	L	C	T	L	C	T	L	C	T	L	C	T
jqlang__jq-2919	F	N/A	N/A	F	N/A	N/A	F	N/A	N/A	F	N/A	N/A
facebook__zstd-1390	F	N/A	N/A	F	N/A	N/A	F	N/A	N/A	F	N/A	N/A
jqlang__jq-2728	S	S	F	S	S	F	F	N/A	N/A	F	N/A	N/A
facebook__zstd-983	F	N/A	N/A	F	N/A	N/A	F	N/A	N/A	F	N/A	N/A
jqlang__jq-3165 [†]	F	N/A	N/A	F	N/A	N/A	F	N/A	N/A	F	N/A	N/A
facebook__zstd-1105	F	N/A	N/A	F	N/A	N/A	F	N/A	N/A	F	N/A	N/A
facebook__zstd-809	F	N/A	N/A	F	N/A	N/A	F	N/A	N/A	F	N/A	N/A
jqlang__jq-2821	S	S	F	S	S	F	S	F	N/A	F	N/A	N/A
jqlang__jq-3161	F	N/A	N/A	F	N/A	N/A	F	N/A	N/A	F	N/A	N/A
facebook__zstd-1243	F	N/A	N/A	F	N/A	N/A	F	N/A	N/A	F	N/A	N/A

Table 2: Per-stage evaluation: **Localization** (file overlap with gold patch), **Compilation** (git apply + build), **Testing** (fail_to_pass resolved).

System	Localization	Compilation	Testing
Qwen3 Blind	15	5	0
Qwen3 Enhanced	40	4	0
GPT-4.1-mini	40	0	0
SWE-bench-AGENT (Ours)	15	9	0

On the redis subset of SWE-bench-C (89 instances), SWE-bench-AGENT produced one clean apply (`redis__redis-8580`), exceeding the V1 Qwen3 Blind and V3 GPT-4.1-mini baselines, which both failed to apply any redis patch. The V2 Qwen3 Enhanced baseline reached five applied patches, but its prompt was augmented with gold-patch metadata (file paths, affected functions, test names) that SWE-bench-AGENT does not consume. The single SWE-bench-AGENT apply subsequently failed compilation due to a macOS-specific `fstat64/stat64` declaration issue in the evaluation environment rather than a defect in the generated patch; on a Linux toolchain the same diff is expected to compile. No system in the study resolved any redis instance, consistent with the published Phase 1 baseline of 0% resolution on SWE-bench-C.

Table 3: Per-stage comparison on the redis subset (89 instances). SWE-bench-AGENT is the first redis run on this set to apply a patch outside the V2 enhanced-prompt baseline.

System	Localization	Compilation	Testing
V1: Qwen3 Blind	0	0	0
V2: Qwen3 Enhanced	5	0	0
V3: GPT-4.1-mini	0	0	0
SWE-bench-AGENT	1	0	0

On the facebook/zstd subset of SWE-bench-C (44 instances), SWE-bench-AGENT correctly localised the gold-patch file in three cases (`facebook__zstd-1230`, `facebook__zstd-2451`, `facebook__zstd-3530`), exceeding the V1 Qwen3 Blind and V3 GPT-4.1-mini baselines, which both failed to localise any zstd instance. The V2 Qwen3 Enhanced baseline reached eight localisations, but its prompt is augmented with gold-patch metadata (file paths, affected functions, test names) that SWE-bench-AGENT does not consume. The three localised SWE-bench-AGENT diffs each targeted the correct file but failed `git apply`'s 3-way merge due to mismatched hunk context lines; no system in the study compiled or resolved any zstd instance

($T = 0$).

Table 4: Per-stage comparison on the facebook/zstd subset (44 instances).

System	Localization	Compilation	Testing
V1: Qwen3 Blind	0	0	0
V2: Qwen3 Enhanced	8	0	0
V3: GPT-4.1-mini	0	0	0
SWE-bench-AGENT	3	0	0

4.3 Analysis

4.3.1 Patch Application Failures

The dominant failure mode in SWE-bench-C under single-shot prompting is unsuccessful patch application or incorrect patch localization. The model has to identify the correct file and generate diffs with context that matches the repository state at `base_commit`. Without direct access to the repository, the model often guesses file structure, function names, or surrounding lines incorrectly, leading to patches that cannot be applied cleanly.

This mirrors our findings on SWE-bench-C: the absence of source context makes localization the first major bottleneck. Our SWE-bench-AGENT pipeline addresses this directly through a code dependency graph and a Localizer that surfaces verbatim file contents in the `ContextBundle`, recovering the gold-patch file on 22 instances across the 174-instance run set and producing one apply-clean patch (`redis_redis-8580`) — confirming that repository-aware retrieval consistently outperforms single-shot guessing on the localization stage.

4.3.2 Applied Patches Often Fail Testing

Even when a patch applies successfully, issue resolution remains difficult. Passing the harness requires more than producing a plausible-looking edit: the change must satisfy the exact repository semantics and test expectations. In many cases, single-shot generated patches address only part of the issue, implement a superficially reasonable workaround, or miss important edge cases encoded in the tests.

This gap between **patch applicability** and **true resolution** is one of the central lessons of SWE-bench-C. A patch that appears correct at a glance may still fail the benchmark because it does not align with the repository’s actual behavior — every system in this study, single-shot or agentic, reached zero resolutions across the 174-instance run set.

4.3.3 Why SWE-bench-C Remains Challenging

The benchmark remains difficult because successful issue resolution depends on several forms of repository-grounded reasoning:

- **Correct localization:** identifying the right file, function, or module from issue text alone.
- **Commit-specific reasoning:** the fix must match the repository exactly at `base_commit`.

- **Behavioral correctness:** the patch must satisfy hidden assumptions encoded in the test suite.
- **Cross-file dependencies:** bug fixes frequently require coordinated changes across multiple components of the codebase.

These requirements make SWE-bench-C fundamentally different from traditional benchmark tasks that can be solved using only local code generation ability.

4.3.4 Comparison with Single-Shot Baselines

Compared with the Phase 1 single-shot baselines, SWE-bench-AGENT reduces the burden of blind localization by replacing prompt-only retrieval with a structured graph traversal. The single-shot baselines suffered most visibly from localization failure followed by compilation failure; our agentic pipeline exceeds even the prompt-engineered V2 Qwen3 Enhanced baseline on localization (22 vs. 17 across the 174-instance run set), despite not consuming any gold-patch metadata, and produces the only apply-clean patch among the four systems on the redis subset. The same core limitation remains across both approaches: **producing patches whose hunk context exactly matches the repository state is the principal barrier to compilation**, and closing this gap is the highest-leverage next step for our system.

4.4 Summary

SWE-bench-AGENT improves on the Phase 1 single-shot baselines at every measured stage prior to resolution:

- **Localisation:** the graph-based Localizer recovers the gold-patch file in 22 of 174 instances without any issue-text filename leakage, exceeding V1 (5), V2 (17), and V3 (1).
- **Patch generation:** one apply-clean diff (`redis_redis-8580`) and structurally valid diffs on most remaining instances confirm that the multi-agent handoff conveys sufficient context to the Patcher.
- **Resolution:** no system, single-shot or agentic, resolved any of the 174 instances; the apply-to-resolve gap remains the benchmark’s central open challenge.

5 Individual Contributions

- **Hithaishi Surendra:** I implemented the ValidatorAgent for compilation and test validation with retry routing. Debugged the pipeline for evaluations and executed baseline evaluations on the jqlang/jq repository using local vLLM on HPUs at Sol.
- **Jahnvi Seth:** I implemented the planner and localizer agents in the codebase to handle the reasoning over the issue text and pipeline control and use the graph and tools to retrieve right files or functions.
- **Samhitha Harish:** I worked on running the SWE-bench-c results and pipelines. I also ran evaluations on the same.

- **Samudyata Sudarshan Jagirdar:** Owned the code-dependency graph layer DepGraph schema, tree-sitter graph builder, and per-repository graph generation and ran the Phase 1 SWE-bench-C single-shot baselines as well as the Phase 2 SWE-bench-AGENT evaluations on redis.
- **Gursparsh Singh Sodhi:** I implemented the Diagnostician and Patcher agents, which analyze localized C source files to identify bug root causes and generate valid unified diff patches with feedback loop retry handling. I also ran baseline evaluations on the redis/redis and facebook/zstd repositories.

References

- [1] C. E. Jimenez *et al.*, “SWE-bench: Can language models resolve real-world GitHub issues?” *arXiv preprint arXiv:2310.06770*, 2023.
- [2] “MSWE-agent,” 2025. [Online]. Available: <https://github.com/multi-swe-bench/MSWE-agent>